MODUL 10&11

DATA STORAGE

1. Mengimplementasikan Basis Data SQLite dan SQL Helper pada Aplikasi Flutter, Serta Menangani Operasi Penyimpanan Data Secara Offline Berbasis CRUD dan Querying..
    a. Code
        - Main.dart

```dart
import 'package:flutter/material.dart';
import 'sql_helper.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'MindFlow - Premium Journal',
      debugShowCheckedModeBanner: false,
      themeMode: ThemeMode.dark, // Standardize to premium dark
mode
      darkTheme: ThemeData(
        brightness: Brightness.dark,
        scaffoldBackgroundColor: const Color(0xFF0C0A1C), // Deep
space black-blue
        colorScheme: const ColorScheme.dark(
          primary: Color(0xFF7C4DFF), // Radiant violet
          secondary: Color(0xFF00E5FF), // Cyan spark
          surface: Color(0xFF161233), // Slate purple
          onPrimary: Colors.white,
          onSecondary: Colors.black,
        ),
        textTheme: const TextTheme(
          displayLarge: TextStyle(fontSize: 32, fontWeight:
FontWeight.bold, color: Colors.white),
          titleLarge: TextStyle(fontSize: 20, fontWeight:
FontWeight.w600, color: Colors.white),
          bodyLarge: TextStyle(fontSize: 16, color:
Color(0xFFCFD8DC)),
          bodyMedium: TextStyle(fontSize: 14, color:
Color(0xFF90A4AE)),
        ),
```

```
        cardTheme: CardThemeData(
          color: const Color(0xFF1E1945),
          elevation: 4,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(16),
            side: const BorderSide(color: Color(0xFF322A75), width:
1),
          ),
        ),
        useMaterial3: true,
      ),
      home: const MyHomePage(title: 'MindFlow'),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});

  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> with
SingleTickerProviderStateMixin {
  // All journal items
  List<Map<String, dynamic>> _journals = [];
  bool _isLoading = true;

  // Search and Filter controller
  final TextEditingController _searchController =
TextEditingController();
  String _searchQuery = '';

  // Input Controllers
  final TextEditingController _titleController =
TextEditingController();
  final TextEditingController _descriptionController =
TextEditingController();
  String _selectedCategory = '📝 Journal';

  final List<String> _categories = [
    '📝 Journal',
    '💡 Idea',
    '💼 Work',
    '📌 Task',
    '✨ Personal'
```

```
  ];

  @override
  void initState() {
    super.initState();
    _refreshJournals();
  }

  // Get data from database
  void _refreshJournals() async {
    try {
      final data = await SQLHelper.getItems();
      setState(() {
        _journals = data;
        _isLoading = false;
      });
    } catch (e, stack) {
      debugPrint("DB ERROR: $e");
      debugPrint(stack.toString());
      setState(() {
        _isLoading = false;
      });
    }
  }

  // Parse category and clean description from saved text
  Map<String, String> _parseDescription(String? fullDesc) {
    if (fullDesc == null || fullDesc.isEmpty) {
      return {'category': '📝 Journal', 'content': ''};
    }

    // Check if description starts with [CategoryName]
    if (fullDesc.startsWith('[') && fullDesc.contains(']')) {
      final closingBracketIndex = fullDesc.indexOf(']');
      final categoryPart = fullDesc.substring(1,
closingBracketIndex);
      final contentPart = fullDesc.substring(closingBracketIndex +
1).trim();
      return {'category': categoryPart, 'content': contentPart};
    }

    return {'category': '📝 Journal', 'content': fullDesc};
  }

  // Create helper method to show custom messages
  void _showNotification(String message, bool isError) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Row(
```

```
          children: [
            Icon(
              isError ? Icons.error_outline :
Icons.check_circle_outline,
              color: isError ? Colors.redAccent :
Colors.greenAccent,
            ),
            const SizedBox(width: 12),
            Expanded(
              child: Text(
                message,
                style: const TextStyle(fontWeight: FontWeight.w500,
fontSize: 14, color: Colors.white),
              ),
            ),
          ],
        ),
        backgroundColor: const Color(0xFF1E1945),
        duration: const Duration(seconds: 3),
        behavior: SnackBarBehavior.floating,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(12),
          side: BorderSide(
            color: isError ? Colors.redAccent.withValues(alpha:
0.3) : const Color(0xFF7C4DFF).withValues(alpha: 0.3),
            width: 1,
          ),
        ),
      ),
    );
  }

  // Add Item to Database
  Future<void> _addItem() async {
    if (_titleController.text.trim().isEmpty) {
      _showNotification('Title cannot be empty!', true);
      return;
    }

    // Save as: [Category]Description
    final combinedDesc =
'[$_selectedCategory]${_descriptionController.text.trim()}';
    await SQLHelper.createItem(_titleController.text.trim(),
combinedDesc);
    _showNotification('New entry added successfully!', false);

    _refreshJournals();
  }
```

```dart
  // Update Item in Database
  Future<void> _updateItem(int id) async {
    if (_titleController.text.trim().isEmpty) {
      _showNotification('Title cannot be empty!', true);
      return;
    }

    final combinedDesc =
'[$_selectedCategory]${_descriptionController.text.trim()}';
    await SQLHelper.updateItem(id, _titleController.text.trim(),
combinedDesc);
    _showNotification('Entry updated successfully!', false);

    _refreshJournals();
  }

  // Delete Item from Database
  Future<void> _deleteItem(int id) async {
    await SQLHelper.deleteItem(id);
    _showNotification('Entry deleted.', false);
    _refreshJournals();
  }

  // Show bottom sheet to add or edit journal
  void _showForm(int? id) async {
    if (id != null) {
      final existingJournal = _journals.firstWhere((element) =>
element['id'] == id);
      _titleController.text = existingJournal['title'];

      final parsed =
_parseDescription(existingJournal['description']);
      _descriptionController.text = parsed['content'] ?? '';

      // Ensure category matches valid category or default
      final savedCategory = parsed['category'] ?? '📝 Journal';
      if (_categories.contains(savedCategory)) {
        _selectedCategory = savedCategory;
      } else {
        _selectedCategory = '📝 Journal';
      }
    } else {
      _titleController.clear();
      _descriptionController.clear();
      _selectedCategory = '📝 Journal';
    }

    showModalBottomSheet(
      context: context,
```

```
      isScrollControlled: true,
      backgroundColor: const Color(0xFF120E2C),
      shape: const RoundedRectangleBorder(
        borderRadius: BorderRadius.only(
          topLeft: Radius.circular(28),
          topRight: Radius.circular(28),
        ),
      ),
      builder: (context) {
        return StatefulBuilder(
          builder: (BuildContext context, StateSetter
setModalState) {
            return Padding(
              padding: EdgeInsets.only(
                top: 24,
                left: 20,
                right: 20,
                bottom: MediaQuery.of(context).viewInsets.bottom +
24,
              ),
              child: SingleChildScrollView(
                child: Column(
                  mainAxisSize: MainAxisSize.min,
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Row(
                      mainAxisAlignment:
MainAxisAlignment.spaceBetween,
                      children: [
                        Text(
                          id == null ? 'Create New Entry' : 'Edit
Journal Entry',
                          style: const TextStyle(
                            fontSize: 22,
                            fontWeight: FontWeight.bold,
                            color: Colors.white,
                          ),
                        ),
                        IconButton(
                          onPressed: () => Navigator.pop(context),
                          icon: const Icon(Icons.close, color:
Color(0xFF90A4AE)),
                        ),
                      ],
                    ),
                    const SizedBox(height: 16),

                    // Title input field
                    TextField(
```

```
                    controller: _titleController,
                    style: const TextStyle(color: Colors.white,
fontSize: 16),
                    decoration: InputDecoration(
                      labelText: 'Title',
                      labelStyle: const TextStyle(color:
Color(0xFF90A4AE)),
                      focusedBorder: OutlineInputBorder(
                        borderRadius: BorderRadius.circular(12),
                        borderSide: const BorderSide(color:
Color(0xFF7C4DFF), width: 2),
                      ),
                      enabledBorder: OutlineInputBorder(
                        borderRadius: BorderRadius.circular(12),
                        borderSide: const BorderSide(color:
Color(0xFF322A75), width: 1.5),
                      ),
                      filled: true,
                      fillColor: const Color(0xFF1E1945),
                    ),
                  ),
                  const SizedBox(height: 16),

                  // Category dropdown selector
                  const Text(
                    'Select Category',
                    style: TextStyle(
                      fontSize: 14,
                      fontWeight: FontWeight.w500,
                      color: Color(0xFFCFD8DC),
                    ),
                  ),
                  const SizedBox(height: 8),
                  Container(
                    padding: const
EdgeInsets.symmetric(horizontal: 16, vertical: 4),
                    decoration: BoxDecoration(
                      color: const Color(0xFF1E1945),
                      borderRadius: BorderRadius.circular(12),
                      border: Border.all(color: const
Color(0xFF322A75), width: 1.5),
                    ),
                    child: DropdownButtonHideUnderline(
                      child: DropdownButton<String>(
                        value: _selectedCategory,
                        dropdownColor: const Color(0xFF120E2C),
                        icon: const
Icon(Icons.keyboard_arrow_down, color: Color(0xFF7C4DFF)),
                        isExpanded: true,
```

```
                        style: const TextStyle(color:
Colors.white, fontSize: 16),
                        items: _categories.map((String value) {
                          return DropdownMenuItem<String>(
                            value: value,
                            child: Text(value),
                          );
                        }).toList(),
                        onChanged: (newValue) {
                          if (newValue != null) {
                            setModalState(() {
                              _selectedCategory = newValue;
                            });
                            setState(() {
                              _selectedCategory = newValue;
                            });
                          }
                        },
                      ),
                    ),
                  ),
                  const SizedBox(height: 16),

                  // Description text area
                  TextField(
                    controller: _descriptionController,
                    style: const TextStyle(color: Colors.white,
fontSize: 15),
                    maxLines: 5,
                    decoration: InputDecoration(
                      labelText: 'Write your thoughts here...',
                      alignLabelWithHint: true,
                      labelStyle: const TextStyle(color:
Color(0xFF90A4AE)),
                      focusedBorder: OutlineInputBorder(
                        borderRadius: BorderRadius.circular(12),
                        borderSide: const BorderSide(color:
Color(0xFF7C4DFF), width: 2),
                      ),
                      enabledBorder: OutlineInputBorder(
                        borderRadius: BorderRadius.circular(12),
                        borderSide: const BorderSide(color:
Color(0xFF322A75), width: 1.5),
                      ),
                      filled: true,
                      fillColor: const Color(0xFF1E1945),
                    ),
                  ),
                  const SizedBox(height: 24),
```

```
                    // Action buttons
                    Row(
                      children: [
                        Expanded(
                          child: ElevatedButton(
                            onPressed: () {
                              Navigator.pop(context);
                            },
                            style: ElevatedButton.styleFrom(
                              backgroundColor: Colors.transparent,
                              foregroundColor: Colors.white,
                              elevation: 0,
                              side: const BorderSide(color:
Color(0xFF322A75), width: 1.5),
                              shape: RoundedRectangleBorder(
                                borderRadius:
BorderRadius.circular(12),
                              ),
                              padding: const
EdgeInsets.symmetric(vertical: 16),
                            ),
                            child: const Text('Cancel'),
                          ),
                        ),
                        const SizedBox(width: 12),
                        Expanded(
                          child: Container(
                            decoration: BoxDecoration(
                              gradient: const LinearGradient(
                                colors: [Color(0xFF7C4DFF),
Color(0xFF00E5FF)],
                                begin: Alignment.topLeft,
                                end: Alignment.bottomRight,
                              ),
                              borderRadius:
BorderRadius.circular(12),
                              boxShadow: [
                                BoxShadow(
                                  color: const
Color(0xFF7C4DFF).withValues(alpha: 0.3),
                                  blurRadius: 10,
                                  offset: const Offset(0, 4),
                                )
                              ],
                            ),
                            child: ElevatedButton(
                              onPressed: () async {
                                if (id == null) {
```

```
                                await _addItem();
                              } else {
                                await _updateItem(id);
                              }
                              if (context.mounted) {
                                Navigator.pop(context);
                              }
                            },
                            style: ElevatedButton.styleFrom(
                              backgroundColor:
Colors.transparent,
                              foregroundColor: Colors.white,
                              shadowColor: Colors.transparent,
                              shape: RoundedRectangleBorder(
                                borderRadius:
BorderRadius.circular(12),
                              ),
                              padding: const
EdgeInsets.symmetric(vertical: 16),
                            ),
                            child: Text(id == null ? 'Add
Entry' : 'Update Entry'),
                          ),
                        ),
                      ),
                    ),
                  ],
                ),
              ],
            ),
          ),
        ),
      );
    },
  );
}

// Get color for categories
Color _getCategoryColor(String category) {
  if (category.contains('💡')) return const Color(0xFFFFC107); //
Amber
  if (category.contains('💼')) return const Color(0xFFFF5722);
// Orange/DeepOrange
  if (category.contains('📌')) return const Color(0xFF00E5FF);
// Cyan
  if (category.contains('✨')) return const Color(0xFFE040FB); //
Magenta
  return const Color(0xFF7C4DFF); // default Violet
}
```

```
  @override
  Widget build(BuildContext context) {
    // Filter journals based on search queries
    final filteredJournals = _journals.where((journal) {
      final title = journal['title'].toString().toLowerCase();
      final desc = journal['description'].toString().toLowerCase();
      final query = _searchQuery.toLowerCase();
      return title.contains(query) || desc.contains(query);
    }).toList();

    // Stats calculations
    final int totalCount = _journals.length;
    final int tasksCount = _journals.where((j) =>
j['description'].toString().startsWith('[📌')).length;
    final int ideasCount = _journals.where((j) =>
j['description'].toString().startsWith('[💡')).length;

    return Scaffold(
      body: SafeArea(
        child: Column(
          children: [
            // Top App Bar with premium Gradient title
            Padding(
              padding: const EdgeInsets.fromLTRB(20, 24, 20, 16),
              child: Row(
                mainAxisAlignment: MainAxisAlignment.spaceBetween,
                children: [
                  Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      ShaderMask(
                        shaderCallback: (bounds) => const
LinearGradient(
                          colors: [Color(0xFF7C4DFF),
Color(0xFF00E5FF)],
                        ).createShader(bounds),
                        child: Text(
                          widget.title,
                          style: const TextStyle(
                            fontSize: 30,
                            fontWeight: FontWeight.bold,
                            color: Colors.white,
                          ),
                        ),
                      ),
                      const SizedBox(height: 4),
                      const Text(
                        'Unlock your flow of thoughts',
```

```
                      style: TextStyle(
                        fontSize: 13,
                        color: Color(0xFF90A4AE),
                        fontWeight: FontWeight.w400,
                      ),
                    ),
                  ],
                ),
                Container(
                  decoration: BoxDecoration(
                    gradient: const LinearGradient(
                      colors: [Color(0xFF7C4DFF),
Color(0xFF00E5FF)],
                    ),
                    borderRadius: BorderRadius.circular(12),
                  ),
                  child: Container(
                    margin: const EdgeInsets.all(1.5),
                    decoration: BoxDecoration(
                      color: const Color(0xFF0C0A1C),
                      borderRadius: BorderRadius.circular(11),
                    ),
                    child: IconButton(
                      icon: const Icon(Icons.add, color:
Color(0xFF00E5FF)),
                      onPressed: () => _showForm(null),
                      tooltip: 'Add Entry',
                    ),
                  ),
                ),
              ],
            ),
          ),

          // Statistics Panel
          if (totalCount > 0)
            Padding(
              padding: const EdgeInsets.symmetric(horizontal: 20,
vertical: 8),
              child: Row(
                children: [
                  // Total Stat card
                  Expanded(
                    child: Container(
                      padding: const EdgeInsets.all(12),
                      decoration: BoxDecoration(
                        color: const Color(0xFF161233),
                        borderRadius: BorderRadius.circular(12),
                        border: Border.all(color: const
```

```
Color(0xFF322A75), width: 1),
                        ),
                        child: Column(
                          crossAxisAlignment:
CrossAxisAlignment.start,
                          children: [
                            const Text('Total Items', style:
TextStyle(fontSize: 11, color: Color(0xFF90A4AE))),
                            const SizedBox(height: 4),
                            Text('$totalCount', style: const
TextStyle(fontSize: 18, fontWeight: FontWeight.bold, color:
Colors.white)),
                          ],
                        ),
                      ),
                    ),
                    const SizedBox(width: 8),
                    // Tasks Stat card
                    Expanded(
                      child: Container(
                        padding: const EdgeInsets.all(12),
                        decoration: BoxDecoration(
                          color: const Color(0xFF161233),
                          borderRadius: BorderRadius.circular(12),
                          border: Border.all(color: const
Color(0xFF322A75), width: 1),
                        ),
                        child: Column(
                          crossAxisAlignment:
CrossAxisAlignment.start,
                          children: [
                            const Text('Tasks', style:
TextStyle(fontSize: 11, color: Color(0xFF90A4AE))),
                            const SizedBox(height: 4),
                            Text('$tasksCount', style: const
TextStyle(fontSize: 18, fontWeight: FontWeight.bold, color:
Color(0xFF00E5FF))),
                          ],
                        ),
                      ),
                    ),
                    const SizedBox(width: 8),
                    // Ideas Stat card
                    Expanded(
                      child: Container(
                        padding: const EdgeInsets.all(12),
                        decoration: BoxDecoration(
                          color: const Color(0xFF161233),
                          borderRadius: BorderRadius.circular(12),
```

```
                        border: Border.all(color: const
Color(0xFF322A75), width: 1),
                      ),
                      child: Column(
                        crossAxisAlignment:
CrossAxisAlignment.start,
                        children: [
                          const Text('Ideas', style:
TextStyle(fontSize: 11, color: Color(0xFF90A4AE))),
                          const SizedBox(height: 4),
                          Text('$ideasCount', style: const
TextStyle(fontSize: 18, fontWeight: FontWeight.bold, color:
Color(0xFFFFC107))),
                        ],
                      ),
                    ),
                  ),
                ],
              ),
            ),

            // Search Bar
            Padding(
              padding: const EdgeInsets.symmetric(horizontal: 20,
vertical: 8),
              child: TextField(
                controller: _searchController,
                onChanged: (value) {
                  setState(() {
                    _searchQuery = value;
                  });
                },
                style: const TextStyle(color: Colors.white,
fontSize: 14),
                decoration: InputDecoration(
                  hintText: 'Search journals...',
                  hintStyle: const TextStyle(color:
Color(0xFF607D8B)),
                  prefixIcon: const Icon(Icons.search, color:
Color(0xFF7C4DFF)),
                  suffixIcon: _searchQuery.isNotEmpty
                      ? IconButton(
                          icon: const Icon(Icons.clear, color:
Color(0xFF90A4AE)),
                          onPressed: () {
                            _searchController.clear();
                            setState(() {
                              _searchQuery = '';
                            });
```

```
                        },
                      )
                    : null,
                filled: true,
                fillColor: const Color(0xFF161233),
                contentPadding: const
EdgeInsets.symmetric(vertical: 0),
                focusedBorder: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(12),
                  borderSide: const BorderSide(color:
Color(0xFF7C4DFF), width: 1.5),
                ),
                enabledBorder: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(12),
                  borderSide: const BorderSide(color:
Color(0xFF322A75), width: 1),
                ),
              ),
            ),
          ),

          const SizedBox(height: 8),

          // Main Content Area
          Expanded(
            child: _isLoading
                ? const Center(
                    child: CircularProgressIndicator(
                      color: Color(0xFF7C4DFF),
                    ),
                  )
                : filteredJournals.isEmpty
                    ? _buildEmptyState()
                    : ListView.builder(
                        padding: const
EdgeInsets.symmetric(horizontal: 20, vertical: 8),
                        itemCount: filteredJournals.length,
                        itemBuilder: (context, index) {
                          final journal =
filteredJournals[index];
                          final int id = journal['id'];
                          final String title = journal['title'];
                          final String fullDesc =
journal['description'] ?? '';
                          final parsed =
_parseDescription(fullDesc);
                          final String category =
parsed['category'] ?? '📝 Journal';
                          final String content =
```

```
parsed['content'] ?? '';
                            final Color catColor =
_getCategoryColor(category);
                            final String timestamp =
journal['createdAt'] ?? '';

                            return Padding(
                              padding: const
EdgeInsets.only(bottom: 14),
                              child: Container(
                                decoration: BoxDecoration(
                                  borderRadius:
BorderRadius.circular(16),
                                  gradient: LinearGradient(
                                    colors: [
                                      const Color(0xFF1E1945),
                                      const
Color(0xFF161233).withValues(alpha: 0.8)
                                    ],
                                    begin: Alignment.topLeft,
                                    end: Alignment.bottomRight,
                                  ),
                                  border: Border.all(
                                    color: const
Color(0xFF322A75).withValues(alpha: 0.8),
                                    width: 1,
                                  ),
                                  boxShadow: [
                                    BoxShadow(
                                      color:
Colors.black.withValues(alpha: 0.2),
                                      blurRadius: 8,
                                      offset: const Offset(0, 4),
                                    )
                                  ],
                                ),
                                child: ClipRRect(
                                  borderRadius:
BorderRadius.circular(16),
                                  child: Theme(
                                    data:
Theme.of(context).copyWith(
                                      dividerColor:
Colors.transparent,
                                    ),
                                    child: ExpansionTile(
                                      leading: CircleAvatar(
                                        backgroundColor:
catColor.withValues(alpha: 0.12),
```

```
                                        child: Text(
                                          category.substring(0, 2),
// Emoji
                                          style: const
TextStyle(fontSize: 16),
                                        ),
                                      ),
                                      title: Text(
                                        title,
                                        style: const TextStyle(
                                          fontSize: 16,
                                          fontWeight:
FontWeight.bold,
                                          color: Colors.white,
                                        ),
                                      ),
                                      subtitle: Padding(
                                        padding: const
EdgeInsets.only(top: 4),
                                        child: Row(
                                          children: [
                                            // Category tag
                                            Container(
                                              padding: const
EdgeInsets.symmetric(horizontal: 8, vertical: 2),
                                              decoration:
BoxDecoration(
                                                color:
catColor.withValues(alpha: 0.15),
                                                borderRadius:
BorderRadius.circular(6),
                                                border:
Border.all(color: catColor.withValues(alpha: 0.4), width: 0.8),
                                              ),
                                              child: Text(
                                                category.substring(
2),
                                                style: TextStyle(
                                                  color: catColor,
                                                  fontSize: 10,
                                                  fontWeight:
FontWeight.bold,
                                                ),
                                              ),
                                            ),
                                            const SizedBox(width:
8),
                                            // Date Text
                                            Expanded(
```

```
                                        child: Text(
                                          _formatTimestamp(ti
mestamp),
                                          style: const
TextStyle(
                                            color:
Color(0xFF90A4AE),
                                            fontSize: 11,
                                          ),
                                          overflow:
TextOverflow.ellipsis,
                                        ),
                                      ),
                                    ],
                                  ),
                                ),
                                trailing: const Icon(
                                  Icons.keyboard_arrow_down,
                                  color: Color(0xFF90A4AE),
                                ),
                                children: [
                                  Padding(
                                    padding: const
EdgeInsets.fromLTRB(16, 4, 16, 16),
                                    child: Column(
                                      crossAxisAlignment:
CrossAxisAlignment.start,
                                      children: [
                                        const Divider(color:
Color(0xFF322A75), height: 16),
                                        const
SizedBox(height: 4),
                                        Text(
                                          content.isEmpty
                                              ? 'No
description added for this entry.'
                                              : content,
                                          style: const
TextStyle(
                                            color:
Color(0xFFECEFF1),
                                            fontSize: 14,
                                            height: 1.4,
                                          ),
                                        ),
                                        const
SizedBox(height: 16),
                                        Row(
                                          mainAxisAlignment:
```

```
MainAxisAlignment.end,
                                                    children: [
                                                      // Edit button
                                                      OutlinedButton.ic
on(
                                                        onPressed: ()
=> _showForm(id),
                                                        icon: const
Icon(Icons.edit_outlined, size: 14),
                                                        label: const
Text('Edit', style: TextStyle(fontSize: 12)),
                                                        style:
OutlinedButton.styleFrom(
                                                          foregroundCol
or: const Color(0xFF00E5FF),
                                                          side: const
BorderSide(color: Color(0xFF322A75)),
                                                          shape:
RoundedRectangleBorder(
                                                            borderRadiu
s: BorderRadius.circular(8),
                                                          ),
                                                          padding:
const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
                                                        ),
                                                      ),
                                                      const
SizedBox(width: 8),
                                                      // Delete button
                                                      OutlinedButton.ic
on(
                                                        onPressed: ()
=> _confirmDelete(id),
                                                        icon: const
Icon(Icons.delete_outline, size: 14),
                                                        label: const
Text('Delete', style: TextStyle(fontSize: 12)),
                                                        style:
OutlinedButton.styleFrom(
                                                          foregroundCol
or: Colors.redAccent,
                                                          side: const
BorderSide(color: Color(0xFF322A75)),
                                                          shape:
RoundedRectangleBorder(
                                                            borderRadiu
s: BorderRadius.circular(8),
                                                          ),
                                                          padding:
```

```
const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
                                                    ),
                                                  ),
                                                ],
                                              ),
                                            ],
                                          ),
                                        ),
                                      ],
                                    ),
                                  ),
                                ),
                              );
                            },
                          ),
                        ),
                      ],
                    ),
                  ),
                  floatingActionButton: FloatingActionButton(
                    onPressed: () => _showForm(null),
                    backgroundColor: const Color(0xFF7C4DFF),
                    foregroundColor: Colors.white,
                    elevation: 6,
                    shape: RoundedRectangleBorder(
                      borderRadius: BorderRadius.circular(16),
                    ),
                    child: const Icon(Icons.add, size: 28),
                  ),
                );
              }

              // Format SQLite timestamp (simplified string extraction)
              String _formatTimestamp(String timestamp) {
                if (timestamp.isEmpty) return '';
                try {
                  // Input formats standard: 2026-06-01 14:38:32 or
DateTime.now().toString()
                  // Let's make a readable short date/time
                  final parsedDate = DateTime.tryParse(timestamp);
                  if (parsedDate != null) {
                    final months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
                    final day = parsedDate.day;
                    final month = months[parsedDate.month - 1];
                    final year = parsedDate.year;
                    final hour = parsedDate.hour.toString().padLeft(2, '0');
                    final minute = parsedDate.minute.toString().padLeft(2,
```

```
'0');
        return '$day $month $year, $hour:$minute';
      }
    } catch (_) {}
    return timestamp;
  }

  // Prompt Dialog to Confirm Deletion
  void _confirmDelete(int id) {
    showDialog(
      context: context,
      builder: (context) {
        return AlertDialog(
          backgroundColor: const Color(0xFF120E2C),
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(20),
            side: const BorderSide(color: Color(0xFF322A75)),
          ),
          title: const Text('Delete Journal?', style:
TextStyle(color: Colors.white, fontWeight: FontWeight.bold)),
          content: const Text(
            'Are you sure you want to permanently erase this entry
from your memory space?',
            style: TextStyle(color: Color(0xFF90A4AE)),
          ),
          actions: [
            TextButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('Keep it', style: TextStyle(color:
Color(0xFF90A4AE))),
            ),
            ElevatedButton(
              onPressed: () {
                _deleteItem(id);
                Navigator.pop(context);
              },
              style: ElevatedButton.styleFrom(
                backgroundColor: Colors.redAccent,
                foregroundColor: Colors.white,
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(10),
                ),
              ),
              child: const Text('Delete'),
            ),
          ],
        );
      },
    );
```

```
  }

  // Beautiful empty state visual
  Widget _buildEmptyState() {
    return Center(
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 40),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            // Custom premium looking placeholder drawing
            Container(
              height: 150,
              width: 150,
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                gradient: LinearGradient(
                  colors: [
                    const Color(0xFF7C4DFF).withValues(alpha: 0.1),
                    const Color(0xFF00E5FF).withValues(alpha: 0.1)
                  ],
                  begin: Alignment.topLeft,
                  end: Alignment.bottomRight,
                ),
              ),
              child: ShaderMask(
                shaderCallback: (bounds) => const LinearGradient(
                  colors: [Color(0xFF7C4DFF), Color(0xFF00E5FF)],
                ).createShader(bounds),
                child: const Icon(
                  Icons.menu_book_outlined,
                  size: 72,
                  color: Colors.white,
                ),
              ),
            ),
            const SizedBox(height: 24),
            const Text(
              'Your stream is empty',
              style: TextStyle(
                fontSize: 20,
                fontWeight: FontWeight.bold,
                color: Colors.white,
              ),
            ),
            const SizedBox(height: 12),
            const Text(
              'No matching journal entries found. Open your mind,
tap the "+" button, and record your thoughts, ideas, tasks, or work
```

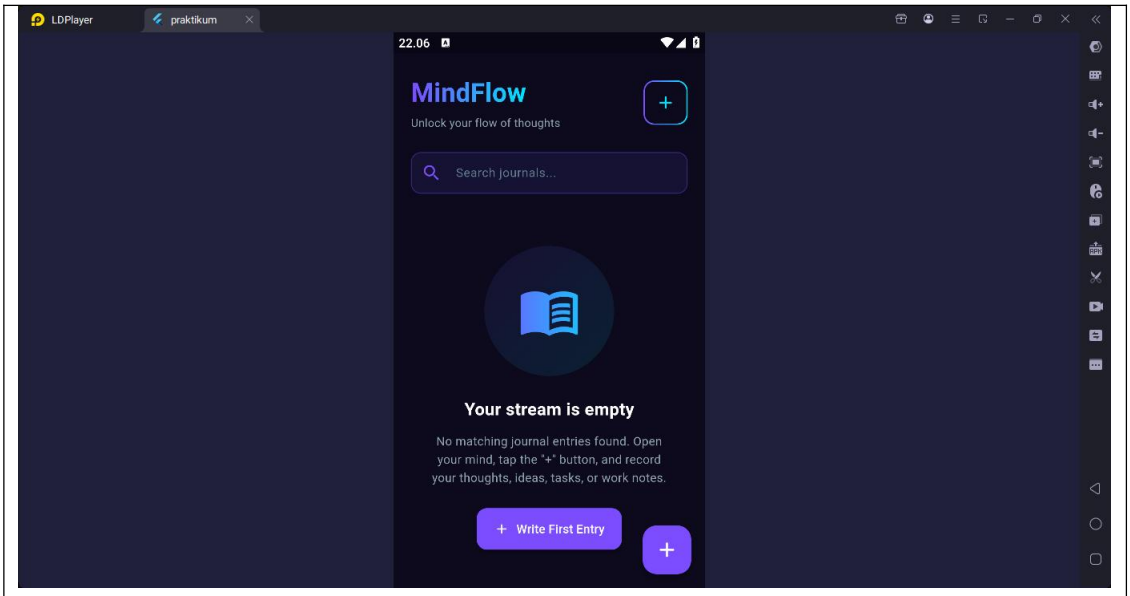
```
notes.',
              textAlign: TextAlign.center,
              style: TextStyle(
                fontSize: 14,
                color: Color(0xFF90A4AE),
                height: 1.5,
              ),
            ),
            const SizedBox(height: 24),
            // Call to action button
            ElevatedButton.icon(
              onPressed: () => _showForm(null),
              icon: const Icon(Icons.add),
              label: const Text('Write First Entry'),
              style: ElevatedButton.styleFrom(
                backgroundColor: const Color(0xFF7C4DFF),
                foregroundColor: Colors.white,
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(12),
                ),
                padding: const EdgeInsets.symmetric(horizontal: 20,
vertical: 14),
                elevation: 4,
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

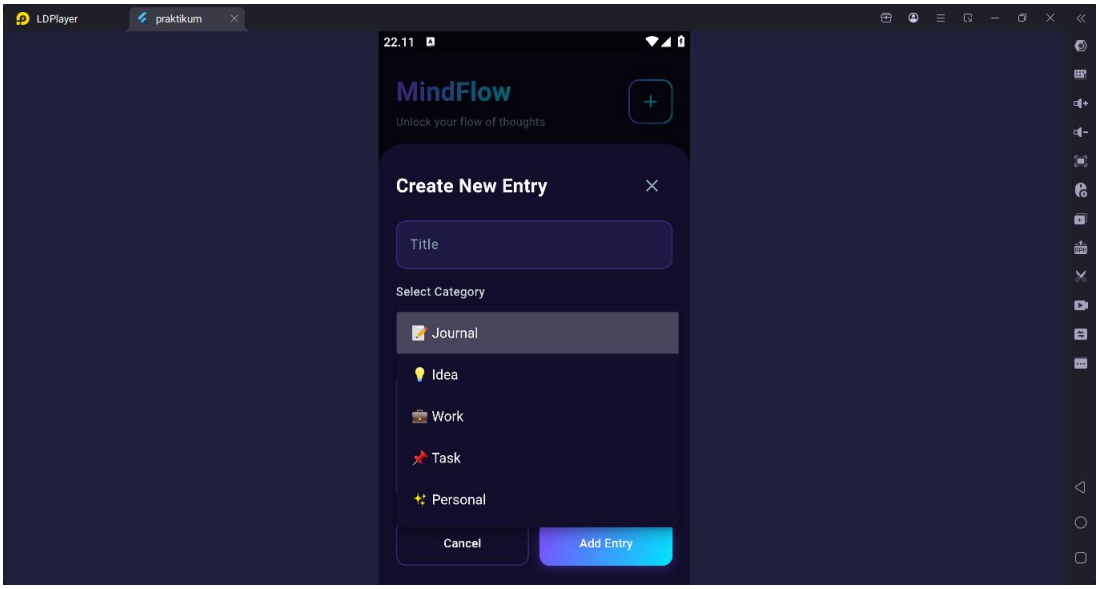
- sql_helper.dart

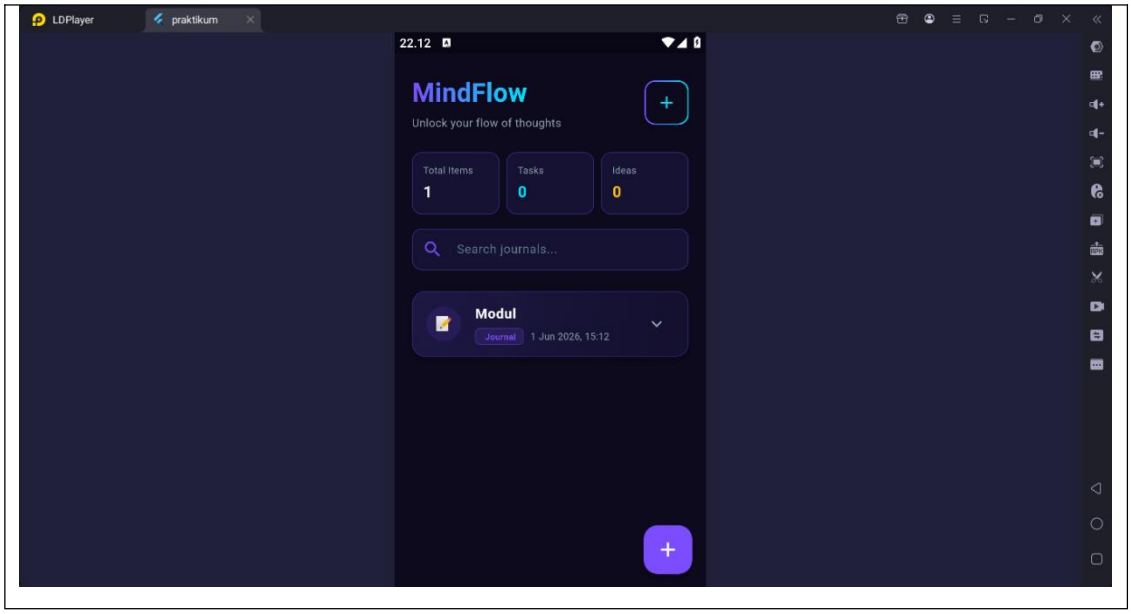
```dart
iimport 'package:flutter/foundation.dart';
import 'package:sqflite/sqflite.dart' as sql;

class SQLHelper {
  // Create tables in database
  static Future<void> createTables(sql.Database database)
async {
    await database.execute("""CREATE TABLE items(
        id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
        title TEXT,
        description TEXT,
        createdAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
      )""");
  }

  // Open database connection
  static Future<sql.Database> db() async {
    return sql.openDatabase(
      'kindacode.db',
      version: 1,
      onCreate: (sql.Database database, int version) async {
        await createTables(database);
      },
    );
  }

  // Create new item
  static Future<int> createItem(String title, String?
description) async {
    final db = await SQLHelper.db();
    final data = {'title': title, 'description': description};
    final id = await db.insert('items', data,
        conflictAlgorithm: sql.ConflictAlgorithm.replace);
    return id;
  }

  // Read all items
  static Future<List<Map<String, dynamic>>> getItems() async {
    final db = await SQLHelper.db();
    return db.query('items', orderBy: "id");
  }

  // Read single item by id
  static Future<List<Map<String, dynamic>>> getItem(int id)
async {
    final db = await SQLHelper.db();
    return db.query('items', where: "id = ?", whereArgs: [id],
limit: 1);
```

```
  }

  // Update item
  static Future<int> updateItem(
      int id, String title, String? description) async {
    final db = await SQLHelper.db();

    final data = {
      'title': title,
      'description': description,
      'createdAt': DateTime.now().toString()
    };

    final result =
        await db.update('items', data, where: "id = ?",
whereArgs: [id]);
    return result;
  }

  // Delete item
  static Future<void> deleteItem(int id) async {
    final db = await SQLHelper.db();
    try {
      await db.delete("items", where: "id = ?", whereArgs:
[id]);
    } catch (err) {
      debugPrint("Something went wrong when deleting an item:
$err");
    }
  }
}
```

b. *Screenshot output*






LDPlayer
praktikum
22.12
MindFlow
Unlock your flow of thoughts
Total Items
1
Tasks
0
Ideas
0
Search journals...
Modul
Journal
1 Jun 2026, 15:12

c. Penjelasan dari code dan output (korelasi)

- Aplikasi Flutter ini dibangun dengan struktur yang teratur untuk mengelola penyimpanan data lokal secara offline menggunakan basis data SQLite. Komponen pertama adalah file sql_helper.dart yang mendefinisikan kelas SQLHelper. Di dalam kelas ini, terdapat fungsi createTables yang bertugas mengeksekusi perintah SQL untuk membuat tabel bernama items dengan atribut berupa id (sebagai *primary key* otomatis), title berbentuk teks, description berbentuk teks, dan createdAt sebagai penanda waktu pembuatan data. Kelas helper ini juga menyediakan metode inisialisasi basis data kindacode.db serta fungsi-fungsi manipulasi data asinkronus seperti pembacaan data (query), pembuatan data baru (insert), pembaruan data (update), dan penghapusan data (delete).
- Selanjutnya, alur antarmuka dan logika interaksi pengguna dikendalikan oleh file main.dart yang bertindak sebagai *entry point* aplikasi. Aplikasi ini mengusung tema bernuansa gelap premium (*premium dark mode*) dengan kombinasi warna ungu violet dan aksen cyan spark. Di halaman utama (MyHomePage), aplikasi menginisialisasi fungsi _refreshJournals() saat aplikasi pertama kali dimuat guna membaca seluruh data yang tersimpan di dalam basis data melalui metode SQLHelper.getItems(). Data hasil pembacaan tersebut kemudian dimasukkan ke dalam variabel _journals untuk diperbarui statusnya (*state*) secara dinamis ke layar pengguna.
- Terakhir, interaksi manipulasi data (CRUD) dimanifestasikan melalui antarmuka pengguna dalam bentuk formulir lembar bawah (*bottom sheet*) yang dipicu oleh fungsi _showForm(). Ketika pengguna ingin menambahkan catatan baru atau mengubah entri yang sudah ada, formulir ini menyediakan bidang masukan (*text field*) untuk judul dan konten, serta menu pilihan (*dropdown*) untuk kategori seperti jurnal, ide, pekerjaan, tugas, atau personal. Saat tombol simpan ditekan, fungsi _addItem() atau _updateItem() akan dijalankan untuk menyimpan perubahan ke basis data , yang kemudian secara otomatis memicu fungsi _showNotification() untuk memunculkan bilah informasi (*SnackBar*) di bagian bawah layar sebagai konfirmasi visual (output) sukses atau gagalnya operasi data lokal tersebut kepada pengguna.